1.0G-RC1 — Tagged Evidence Pack Submission Directive

Daniel Macharia <danielmacharia43@gmail.com>
To: <daniel@ics.ac.ke>
Sat, 11 Jul at 12:37

YOHPAL LIVE RELEASE 1.0G-RC1 — Tagged Evidence Pack Submission Directive

The development team is hereby required to submit the complete **RC1 Evidence Pack against one immutable Git commit**.

No new feature, implementation phase, refactor, or unrelated enhancement may be introduced until the Release Review Board issues a final decision.

The latest verified report confirms that Release 1.0F passed **32/32 tests** and implemented all ten planned patch sections, but the work remained uncommitted, untagged, and untested on Android or iPhone. Crashlytics, password-reset delivery, SSO, LiveKit recovery, and other device-dependent functions were therefore not independently proven.

report F-G .pdf

1. Mandatory repository identity

Developers must return:

```
Release branch:
release/yohpal-live-1.0g-rc1

Full commit hash:
40-character SHA

Annotated tag:
yohpal-live-v1.0.0-rc1

Working tree:
CLEAN

CI run:
Must reference the same commit and tag
```

Required commands:

```
git status --short
git branch --show-current
git rev-parse HEAD
git show --no-patch --decorate HEAD
git tag --points-at HEAD
```

A dirty working tree, abbreviated commit hash, lightweight undocumented tag, or evidence generated from a different commit automatically results in **NO-GO**.

2. Required evidence-pack structure

```
release-evidence/
└── yohpal-live-1.0g-rc1/
    ├── manifest.json
    ├── 01-repository/
    │   ├── branch.txt
    │   ├── commit-hash.txt
    │   ├── tag.txt
    │   ├── git-status.txt
    │   └── commit-diff.txt
    ├── 02-ci/
    │   ├── flutter-analyze.txt
    │   ├── flutter-test.txt
    │   ├── coverage-summary.txt
    │   ├── route-integrity.txt
    │   ├── functions-build.txt
    │   └── functions-test.txt
    ├── 03-android/
```

```
│   ├── report.json
│   ├── screenshots/
│   ├── videos/
│   └── logs/
├── 04-ios/
│   ├── report.json
│   ├── screenshots/
│   ├── videos/
│   └── logs/
├── 05-observability/
│   ├── android-crashlytics.png
│   ├── ios-crashlytics.png
│   └── health-summary.json
├── 06-auth/
│   ├── password-reset/
│   ├── google-sign-in/
│   ├── apple-sign-in/
│   └── second-account-isolation/
├── 07-security/
│   ├── rewarded-ad-ssv-proof.txt
│   ├── environment-config-proof.txt
│   ├── firestore-rules-tests.txt
│   └── security-review.md
├── 08-high-findings/
│   ├── ai-publish-gate/
│   ├── avatar-upload/
│   ├── chat-retry/
│   └── logout-cache-clearing/
├── 09-readiness/
│   ├── updated-production-readiness-review.pdf
│   └── closure-register.json
└── 10-governance/
    ├── release-review-board-resolution.md
    └── signatures.pdf
```

3. Required Codebase

tool/release_certification/rc1_manifest.dart

```dart
import 'dart:convert';
import 'dart:io';

enum GateStatus {
  pending,
  passed,
  failed,
  acceptedRisk,
}

final class CertificationGate {
  const CertificationGate({
    required this.id,
    required this.title,
    required this.severity,
    required this.status,
    required this.evidencePaths,
    this.owner,
    this.deadline,
    this.notes,
  });

  final String id;
  final String title;
  final String severity;
  final GateStatus status;
  final List<String> evidencePaths;
  final String? owner;
  final DateTime? deadline;
  final String? notes;

  factory CertificationGate.fromJson(Map<String, dynamic> json) {
    return CertificationGate(
      id: json['id'] as String,
      title: json['title'] as String,
      severity: json['severity'] as String,
      status: GateStatus.values.byName(json['status'] as String),
      evidencePaths: List<String>.from(
        json['evidencePaths'] as List<dynamic>,
      ),
```

```
      owner: json['owner'] as String?,
      deadline: json['deadline'] == null
          ? null
          : DateTime.parse(json['deadline'] as String),
      notes: json['notes'] as String?,
    );
  }
}

final class Rc1Manifest {
  const Rc1Manifest({
    required this.release,
    required this.branch,
    required this.commitHash,
    required this.tag,
    required this.ciRun,
    required this.gates,
  });

  final String release;
  final String branch;
  final String commitHash;
  final String tag;
  final String ciRun;
  final List<CertificationGate> gates;

  factory Rc1Manifest.fromJson(Map<String, dynamic> json) {
    return Rc1Manifest(
      release: json['release'] as String,
      branch: json['branch'] as String,
      commitHash: json['commitHash'] as String,
      tag: json['tag'] as String,
      ciRun: json['ciRun'] as String,
      gates: (json['gates'] as List<dynamic>)
          .map(
            (entry) => CertificationGate.fromJson(
              entry as Map<String, dynamic>,
            ),
          )
          .toList(growable: false),
    );
  }
}

void main(List<String> args) {
  if (args.length != 1) {
    stderr.writeln(
      'Usage: dart run tool/release_certification/'
      'rc1_manifest.dart <manifest.json>',
    );
    exitCode = 64;
    return;
  }

  final manifestFile = File(args.first);

  if (!manifestFile.existsSync()) {
    stderr.writeln('Manifest not found: ${manifestFile.path}');
    exitCode = 66;
    return;
  }

  final manifest = Rc1Manifest.fromJson(
    jsonDecode(manifestFile.readAsStringSync())
        as Map<String, dynamic>,
  );

  final errors = <String>[];

  if (manifest.branch != 'release/yohpal-live-1.0g-rc1') {
    errors.add('Unexpected release branch: ${manifest.branch}');
  }

  if (!RegExp(r'^[0-9a-f]{40}$').hasMatch(manifest.commitHash)) {
    errors.add('Commit hash must be a full 40-character SHA.');
  }

  if (manifest.tag != 'yohpal-live-v1.0.0-rc1') {
    errors.add('Unexpected RC1 tag: ${manifest.tag}');
  }

  for (final gate in manifest.gates) {
    if (gate.evidencePaths.isEmpty) {
      errors.add('${gate.id} has no evidence.');
      continue;
    }
```

```
    for (final evidencePath in gate.evidencePaths) {
      final exists = File(evidencePath).existsSync() ||
          Directory(evidencePath).existsSync();

      if (!exists) {
        errors.add('${gate.id} missing evidence: $evidencePath');
      }
    }

    if (gate.severity == 'critical' &&
        gate.status != GateStatus.passed) {
      errors.add(
        '${gate.id} is Critical and has not fully passed.',
      );
    }

    if (gate.status == GateStatus.acceptedRisk &&
        (gate.owner == null || gate.deadline == null)) {
      errors.add(
        '${gate.id} accepted risk lacks owner or deadline.',
      );
    }
  }

  if (errors.isNotEmpty) {
    stderr.writeln('RC1 evidence validation failed:');
    for (final error in errors) {
      stderr.writeln(' - $error');
    }
    exitCode = 1;
    return;
  }

  stdout.writeln(
    'RC1 evidence manifest validated for '
    '${manifest.commitHash}.',
  );
}
```

release-evidence/yohpal-live-1.0g-rc1/manifest.json

```
{
  "release": "YohPal Live 1.0G-RC1",
  "branch": "release/yohpal-live-1.0g-rc1",
  "commitHash": "REPLACE_WITH_FULL_SHA",
  "tag": "yohpal-live-v1.0.0-rc1",
  "ciRun": "REPLACE_WITH_CI_RUN_REFERENCE",
  "gates": [
    {
      "id": "C-CRASHLYTICS",
      "title": "Crash reporting proven on Android and iPhone",
      "severity": "critical",
      "status": "pending",
      "evidencePaths": [
        "release-evidence/yohpal-live-1.0g-rc1/05-observability/android-crashlytics.png",
        "release-evidence/yohpal-live-1.0g-rc1/05-observability/ios-crashlytics.png"
      ]
    },
    {
      "id": "C-PASSWORD-RESET",
      "title": "Password reset completes end to end",
      "severity": "critical",
      "status": "pending",
      "evidencePaths": [
        "release-evidence/yohpal-live-1.0g-rc1/06-auth/password-reset/"
      ]
    },
    {
      "id": "C-ENVIRONMENT",
      "title": "All production endpoints are externally configured",
      "severity": "critical",
      "status": "pending",
      "evidencePaths": [
        "release-evidence/yohpal-live-1.0g-rc1/07-security/environment-config-proof.txt"
      ]
    },
    {
      "id": "C-REWARDED-ADS",
      "title": "Rewarded ads use real SDK and server verification",
      "severity": "critical",
      "status": "pending",
      "evidencePaths": [
        "release-evidence/yohpal-live-1.0g-rc1/07-security/rewarded-ad-ssv-proof.txt"
      ]
    },
    {
```

```
      "id": "H-AI-PUBLISH-GATE",
      "title": "AI publishing checklist blocks invalid publication",
      "severity": "high",
      "status": "pending",
      "evidencePaths": [
        "release-evidence/yohpal-live-1.0g-rc1/08-high-findings/ai-publish-gate/"
      ]
    },
    {
      "id": "H-AVATAR-UPLOAD",
      "title": "Creator avatar uses real upload flow",
      "severity": "high",
      "status": "pending",
      "evidencePaths": [
        "release-evidence/yohpal-live-1.0g-rc1/08-high-findings/avatar-upload/"
      ]
    },
    {
      "id": "H-CHAT-RETRY",
      "title": "Chat send failures are surfaced and retryable",
      "severity": "high",
      "status": "pending",
      "evidencePaths": [
        "release-evidence/yohpal-live-1.0g-rc1/08-high-findings/chat-retry/"
      ]
    },
    {
      "id": "H-SESSION-CLEANUP",
      "title": "Logout clears cross-account state",
      "severity": "high",
      "status": "pending",
      "evidencePaths": [
        "release-evidence/yohpal-live-1.0g-rc1/08-high-findings/logout-cache-clearing/"
      ]
    }
  ]
}
```

4. Immutable-commit verification script

scripts/verify_rc1_identity.sh

```
#!/usr/bin/env bash
set -euo pipefail

EXPECTED_BRANCH="release/yohpal-live-1.0g-rc1"
EXPECTED_TAG="yohpal-live-v1.0.0-rc1"

CURRENT_BRANCH="$(git branch --show-current)"
CURRENT_COMMIT="$(git rev-parse HEAD)"
TAG_AT_HEAD="$(git tag --points-at HEAD)"

if [[ "$CURRENT_BRANCH" != "$EXPECTED_BRANCH" ]]; then
  echo "Wrong branch: $CURRENT_BRANCH"
  exit 1
fi

if [[ -n "$(git status --porcelain)" ]]; then
  echo "Working tree is not clean."
  git status --short
  exit 2
fi

if ! grep -qx "$EXPECTED_TAG" <<< "$TAG_AT_HEAD"; then
  echo "Expected tag $EXPECTED_TAG does not point to HEAD."
  exit 3
fi

if [[ ${#CURRENT_COMMIT} -ne 40 ]]; then
  echo "Commit hash is not a full SHA."
  exit 4
fi

echo "RC1 identity verified."
echo "Branch: $CURRENT_BRANCH"
echo "Commit: $CURRENT_COMMIT"
echo "Tag: $EXPECTED_TAG"
```

5. Required CI execution

```
flutter clean
flutter pub get
```

```
dart format --output=none --set-exit-if-changed lib test tool
flutter analyze
dart run tool/check_routes.dart
flutter test --coverage

cd functions
npm ci
npm run build
npm test
```

Then:

```
bash scripts/verify_rc1_identity.sh

dart run \
  tool/release_certification/rc1_manifest.dart \
  release-evidence/yohpal-live-1.0g-rc1/manifest.json
```

6. Release Review Board decision rules

GO

Issue only when:

- Every Critical finding is marked **Passed**.
- Every High finding is marked **Passed**.
- Android validation passes.
- iPhone validation passes.
- Crashlytics events are visible for both platforms.
- Password reset and SSO work end to end.
- Rewarded-ad server verification is operational.
- CI is green against the tagged commit.
- Rollback and emergency feature controls are proven.

CONDITIONAL GO

May be issued only when:

- All Critical findings are fully passed.
- No financial, security, privacy, authentication, or data-integrity risk remains.
- At most two non-critical High findings remain.
- Each remaining item has:
  - formal risk acceptance,
  - named owner,
  - deadline,
  - feature flag or bounded exposure.
- Rollout is limited to **1–5%**.

NO-GO

Mandatory when:

- Any Critical item is open, partial, mitigated, or unverified.
- Evidence is not tied to the tagged commit.
- Android or iPhone evidence is missing.
- Crashlytics is not proven.
- Password reset or SSO fails.
- Reward crediting remains client-trusted.
- The working tree is dirty.
- Required signatures are missing.

7. Official developer submission notice

```
SUBJECT:
YohPal Live 1.0G-RC1 — Mandatory Tagged Evidence Pack Submission

The development team must submit the complete YohPal Live 1.0G-RC1 Evidence Pack against one immutable Git commit.

No additional features, implementation phases, or unrelated refactors are authorized.

Submission must include:

1. Release branch
2. Full commit hash
3. Annotated RC1 tag
```

```
4. Clean working-tree proof
5. CI run tied to the same commit
6. Android regression evidence
7. iPhone regression evidence
8. Crashlytics proof
9. Password-reset proof
10. Google and Apple SSO proof
11. Rewarded-ad SSV proof
12. Environment-config proof
13. Four High-finding closure proofs
14. Updated Production Readiness Review
15. Signed Release Review Board resolution

Incomplete, inconsistent, unsigned, or non-immutable evidence will result in NO-GO.
```

Current executive position

General Availability: NO-GO pending evidence.

Controlled RC1 validation: GO.

No further implementation release should be opened. The next governance action is the Release Review Board review of the submitted tagged evidence pack.

Next Smart Move

Upon receipt of the immutable RC1 Evidence Pack, execute the final Release Review Board session and issue exactly one decision: **GO**, **CONDITIONAL GO**, or **NO-GO**.